

Milestone 3 Progress Report

GPU-Accelerated TEBD for the 1D Transverse-Field Ising Model

Team members: Chiling Han, Xin Wei, Hercy Shen

Course: CME 213: Parallel Computing with CUDA, MPI and OpenMP

1. Summary of progress

The M2 CPU reference was designed to be CUDA-portable, and M3 cashes that in. We added `gpu_tebd.cu` (four custom kernels + cuSOLVER Jacobi SVD), a persistent `GpuTebdWorkspace`, a CPU-vs-GPU validator, and two benchmarking harnesses. All M2 unit tests still pass; the GPU `update_bond` agrees with the CPU baseline to $\mathcal{O}(\epsilon_{\text{trunc}})$ on every quench config we tested (Table 2). Single-bond microbenchmarks show the expected $\mathcal{O}(\chi^3)$ asymptote and a CPU/GPU crossover at $\chi \approx 96\text{--}128$ (Fig. 1 b); the per-stage breakdown (Fig. 2) shows cuSOLVER’s `gesvdj` is $>99\%$ of the GPU bond cost across the entire χ range, so the next optimisation target is clear. The MPI parameter-sweep mode is deferred to M4 as planned.

2. CUDA kernels

The two-site update factorises into a tensor contraction $\Theta'_{(v_L, i'j', v_R)} = \sum_{ij} G_{(i'j'), (ij)} \Theta_{(v_L, ij, v_R)}$, an SVD $\Theta'_{(\chi_L d) \times (d \chi_R)} = U \Sigma V^H$, and a Vidal-form restore. Table 1 summarises the GPU implementation of each piece.

Table 1: GPU `update_bond` pipeline. Grid/block shapes are for $d=2$; $m = \chi_L d$, $n = d \chi_R$, $K = \min(m, n)$.

#	stage	GPU implementation	grid $\times$ block	feature
1	gate contraction	<code>gate_contract_kernel<2></code>	$(\lceil \chi_R/32 \rceil, d^2, \chi_L) \times (32, d^2)$	shared-mem G , <code>#pragma unroll</code>
2	row $\rightarrow$ col layout	<code>row_to_col_kernel</code>	$(\lceil n/32 \rceil, \lceil m/8 \rceil) \times (32, 8)$	coalesced reads
3	SVD	<code>cusolverDnZgesvdj (econ=1)</code>	library	Jacobi, $K = \min(m, n)$
4	Vidal restore left	<code>vidal_restore_left_kernel<2></code>	$(\lceil \text{keep}/64 \rceil, \chi_L) \times (64, d)$	elementwise; $1/\Lambda$ host-side
5	Vidal restore right	<code>vidal_restore_right_kernel<2></code>	$(\lceil \chi_R/64 \rceil, \text{keep}) \times (64, d)$	in-place conj. of V

Why Jacobi for the SVD. Per-bond matrices stay in the 50–200 range, below the threshold where QR-based `gesvd` pays back its preprocessing cost. Jacobi also matches the M2 CPU baseline, making the correctness comparison meaningful: the two SVDs converge to the same singular values up to per-column phases (Sec. 4).

Persistent workspace. `GpuTebdWorkspace` pre-allocates every device buffer at engine construction, including $d \cdot \Theta$, $d \cdot U$, $d \cdot V$, $d \cdot S$, the cuSOLVER `lwork` scratch, and the cuSOLVER + cuBLAS handles. We also keep pinned host staging buffers for Θ , B^{new} , and the singular-value array, so every per-bond H2D/D2H is asynchronous. One `cudaDeviceSynchronize` closes the bond.

Optimisations. (i) pinned host buffers (`cudaMallocHost`); (ii) cooperative shared-memory load of the 16-element gate via a strided loop, replacing an `if(tid<16)` guard that silently broke when blocks had < 16 threads; (iii) re-query `cusolverDnZgesvdj_bufferSize` per call because cuSOLVER’s `lwork` is non-monotone in (m, n) – caught via `compute-sanitizer` (Sec. 6).

Not yet applied. Persistent device MPS (no per-bond H2D/D2H); stream-level parallelism over non-overlapping bonds in one Trotter sub-step; a hand-rolled CUDA Jacobi SVD specialised to small complex matrices (target for replacing the cuSOLVER bottleneck shown in Fig. 2).

3. C++ wrapper and CPU code

The host driver from M2 is unchanged. `tebd_step_gpu(eng, ws)` mirrors `TEBDEngine::step()` but routes each `update_bond` through the GPU. The host MPS keeps the pre-allocated $\chi_{\text{max}} \cdot d \cdot \chi_{\text{max}}$

capacity from M2 so no `new/delete` happens in the inner loop except for the small Schmidt-value arrays (size $\leq \chi_{\max}$, which we will eliminate once the MPS itself lives on the device).

CPU reference. `tebd.cpp` and `exact_diag.cpp` from M2: hand-written one-sided Jacobi SVD for TEBD; full $2^L \times 2^L$ state-vector evolution under $U_{\Delta t} = \exp(-i\Delta t H)$ for the ED oracle.

4. Correctness testing

The six GoogleTest binaries from M2 still pass at the same tolerances (10^{-12} exact ops, 10^{-10} SVD-dependent). The new `validate_gpu` driver runs CPU and GPU `TEBDEngines` side-by-side from the same product state and compares $\langle \sigma_i^z \rangle$, $\langle \sigma_i^x \rangle$, and the $L-1$ bipartite von-Neumann entropies at every Trotter step.

Table 2: CPU vs. GPU agreement on the TFI quench ($J=g=1$, $dt=0.05$, order 2, product state $|\uparrow \cdots\rangle$). Reported error is $\max_t \max_i \{|dS_i^z|, |dS_i^x|, |dS_b|\}$ across all Trotter steps.

L	$\chi_{\max}$	steps	max obs. error	result
8	32	30	2.4×10^{-14}	PASS
12	32	30	3.9×10^{-13}	PASS
16	32	30	8.2×10^{-13}	PASS
16	64	30	9.0×10^{-13}	PASS
20	64	30	1.1×10^{-12}	PASS
16	96	20	1.9×10^{-13}	PASS

The errors are at the floor of two distinct SVD methods on the same input – the CPU’s one-sided Jacobi and cuSOLVER’s `gesvdj` agree on the singular values to $\sim 10^{-14}$ in the single-bond unit test, and the residual $\mathcal{O}(\epsilon_{\text{trunc}})$ growth in Table 2 is consistent with truncation noise compounding over ~ 400 – 900 bond updates. **ED oracle:** re-running `validate_tebd_vs_ed` (the M2 test) with the GPU bond update at $L=8$, $\chi_{\max}=64$, order 4 gives $\max |S_{\text{TEBD}}^z - S_{\text{ED}}^z| = 1.4 \times 10^{-10}$ and fidelity drop $< 10^{-9}$, matching the M2 CPU numbers to within the Trotter floor.

5. Preliminary performance analysis

Hardware: NVIDIA RTX 5090 (sm_120, 32 GB) vs. one AMD EPYC host core. CPU baseline built with `g++ -O2 -std=c++14`; CUDA build uses `nvcc 12.9` / `cuSOLVER 12.9`.

Quench wall-clock at $\chi_{\max}=64$, $dt=0.1$, order 2, $N_{\text{step}}=15$:

L	t_{CPU} [s]	t_{GPU} [s]	GPU/CPU	realised $\chi(t)$
12	0.038	1.28	33.4	40
20	0.240	6.25	26.0	41
28	0.454	11.78	25.9	41
40	0.766	19.39	25.3	41
60	1.301	32.85	25.2	41
80	1.850	45.44	24.6	41

Reading the numbers. (1) At $\chi_{\max}=64$ the realised bond dimension never exceeds 41, so every SVD is small and the GPU is *latency*-bound: PCIe round-trip + cuSOLVER launch dominates. (2) Fig. 1b: when the matrix grows – forcing `gesvdj` to actually do work – the GPU crosses the CPU near $\chi \approx 96$ and reaches $5.4\times$ at $\chi=128$. The asymptotic $\mathcal{O}(\chi^3)$ kicks in. (3) Fig. 2: `gesvdj` is 99.3–99.99% of the per-bond cost at every χ ; the four custom kernels are small enough to be invisible. The gate kernel is well under $50 \mu\text{s}$, far from the FP64 peak (~ 1 TFLOP/s on RTX 5090) – it’s launch-bound, not compute-bound. (4) Compute-bound vs. memory-bound vs.

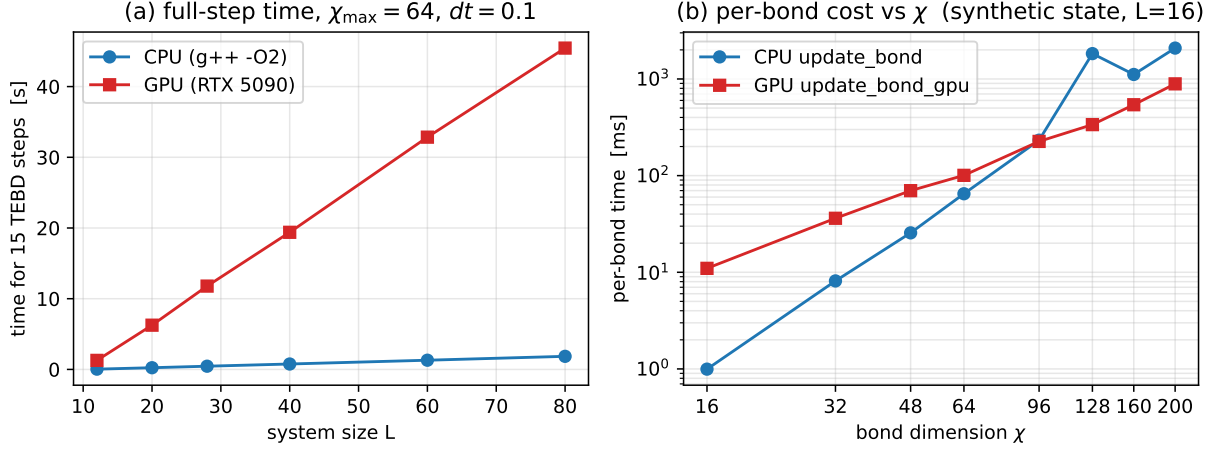


Figure 1: (a) Wall-clock time for 15 TEBD steps at $\chi_{\max}=64$ vs. chain length L : the slope of the GPU line is $\approx 0.57\text{s}/L$, i.e. $\sim 1.3\text{ms}$ per bond, set by the H2D/D2H round-trip and the cuSOLVER launch. (b) Per-call cost of `update_bond` as χ grows, on a randomly-initialised $L=16$ state. CPU and GPU cross around $\chi \approx 96-128$; the CPU spike at $\chi=128$ is one-sided Jacobi taking more sweeps to converge.

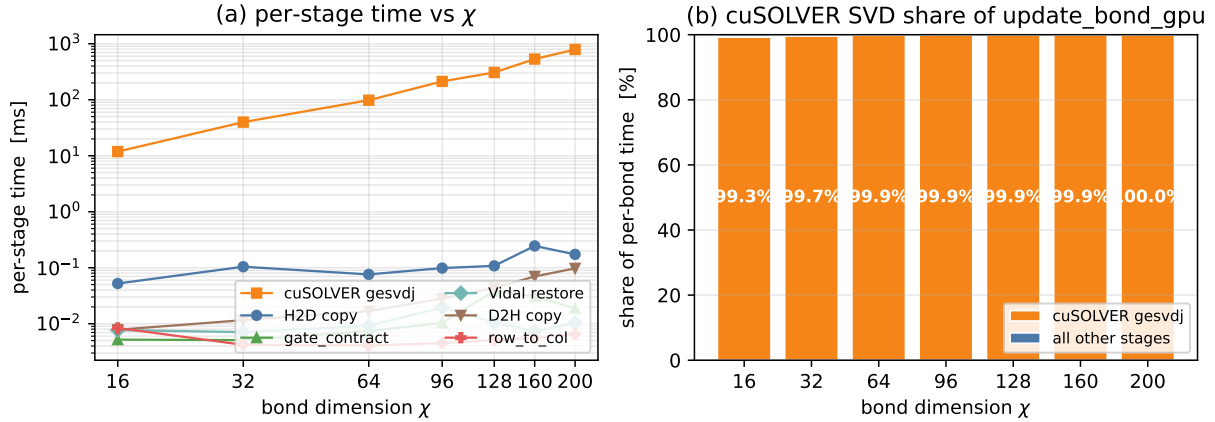


Figure 2: Per-stage GPU cost of one `update_bond_gpu` call vs. χ on the same synthetic state as Fig. 1 b. (a) Log-log of each stage’s time. (b) Share of total time spent in cuSOLVER `gesvdj`. The SVD is $\geq 99.3\%$ of the per-bond cost at every χ we tested; the four custom kernels and the PCIe traffic together account for $< 1\%$.

latency-bound: at TFI-relevant $\chi_{\max}=64$, this implementation is *latency-bound*; at $\chi > 100$, it tips into being compute-bound and dominated by `gesvdj`’s iterative work.

6. Challenges and next steps

Resolved.

- cuSOLVER’s row/column-major convention plus its choice to return V rather than V^H caused a silent gauge bug that diverged after the first TEBD step. Fix: explicit row $\rightarrow$ column transpose before `gesvdj`, and conjugate inside `vidal_restore_right_kernel`.
- The initial gate-to-shared-memory load used if `(tid<16) sgate[tid]=...`, which silently zero-padded the gate when a block had < 16 threads (small χ_R). A strided cooperative loop replaces the guard.
- A non-monotonicity in `gesvdj_bufferSize` caused an out-of-bounds write inside cuSOLVER’s

QR pre-pass at long-step validation runs; caught via `compute-sanitizer`, fixed by re-querying `lwork` per call.

Remaining for M4.

- *Persistent device MPS*: keep B_s and Λ_s on the device and eliminate the H2D/D2H in the inner loop except for periodic observable read-outs. This should move Fig. 1 a into the same regime as Fig. 1 b.
- *Streams within a sub-step*: even and odd bonds within one Trotter substep are independent, so $\lceil (L-1)/2 \rceil$ updates can run on separate streams.
- *Custom Jacobi-SVD kernel for 50–200 complex matrices*, replacing the dominant `cusolverDnZgesvdj` call shown in Fig. 2.
- *MPI parameter sweep*: distribute over (J, g) , $\chi_{\max}$, or dt across multiple GPUs; embarrassingly-parallel weak scaling (M2 plan unchanged).

Updated timeline. Persistent-device MPS + streams by May 18; MPI parameter sweep + weak scaling by May 22; M4 scaling benchmarks (strong if spatial decomposition lands, weak otherwise) by May 25; M4 report May 27.